# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

| | | | |
|---|---|---|---|
| **Course Code:** | CSA06 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO1 – Precision (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 1 – Application Based Questions |
| **Weightage:** | 40% of CIA | **Total Marks:** | 100 (2 Questions × 50 Marks) |
| **CO1 Statement:** | *Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)* | | |
| **Student Name:** | Sameer Ahmed G | **Reg. No.:** | 192421154 |
| **Section / Batch:** | B Tech IT | **Date:** | 16-07-2026 |

**Instructions to Students**

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

| Question Type | Focus Area | Questions | Marks |
|---|---|---|---|
| **Application-Based Questions** | Apply Master Theorem and asymptotic analysis to real-world scenarios | Q1 – Q2 | Q1 –50<br>Q2 –50 |
| | **GRAND TOTAL:** | **100 Marks** | |

## SECTION A – APPLICATION BASED QUESTIONS

**Q1. A routing system processes nodes using the recurrence T(n) = 3T(n/2) + n. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and analyze the space complexity.**

**Q2. A distributed system processes data blocks using the recurrence T(n) = T(n/2) + n. Recall recurrence solving methods, explain how work grows across recursive calls, and determine the asymptotic time complexity. Also analyze the space complexity of the algorithm.**

**Code of Conduct**

*I Sameer Ahmed G/192421154 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Sameer Ahmed G

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20% | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30% | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20% | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| Accuracy of Solution | 20% | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10% | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

**Marks Evaluation:**

| Question No. | Understanding of Problem Context (20%) | Application of Concepts (30%) | Problem-Solving Approach (20%) | Accuracy of Solution (20%) | Clarity (10%) | Total Marks |
|---|---|---|---|---|---|---|
| **Q1** (50 Marks) | | | | | | |
| **Q2** (50 Marks) | | | | | | |
| | | | | | **Overall Total (100)** | |

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

**Q1. Network Routing Optimization System**

A network routing optimization system is responsible for determining efficient communication paths between devices in a large-scale network. Suppose the routing algorithm repeatedly divides the network into two equal subnetworks, recursively optimizes routing in each subnetwork, and finally performs a linear amount of work to merge routing information. The recurrence relation representing this process is:
**T(n) = 3T(n/2) + n**
where:

- **n** = Number of network nodes or routing entries.
- **T(n/2)** = Time required to optimize routing for half of the network.
- **3T(n/2)** = Three independent recursive routing computations.
- **n** = Linear work required to combine routing tables, update path costs, and synchronize routing information.

To determine the time complexity, the **Master Theorem** is applied.

The general form of the Master Theorem is:
**T(n) = aT(n/b) + f(n)**

Comparing the given recurrence:

- **a = 3**
- **b = 2**
- **f(n) = n**

The next step is to compute the critical exponent:

$$n^{\log_b a} = n^{\log_2 3}$$

Since

$$\log_2 3 \approx 1.585$$

therefore,

$$n^{\log_2 3} = n^{1.585}$$

Now compare the functions:

- **f(n) = n = O(n¹)**
- **n¹ grows slower than n¹·⁵⁸⁵**

Thus,

$$f(n) = O\left(n^{\log_2 3-\varepsilon}\right)$$

for some constant **ε > 0**.
This satisfies **Case 1 of the Master Theorem**, which states that if the recursive work dominates the non-recursive work, then

$$T(n) = \Theta\left(n^{\log_b a}\right)$$

Hence,

$$\boxed{T(n) = \Theta(n^{1.585})}$$

This means that the routing computation grows faster than linear time but remains significantly better than quadratic time. The algorithm spends most of its execution time solving the three recursive routing subproblems rather than merging the routing information. Such an approach is suitable for hierarchical routing protocols, distributed path computation, traffic engineering, and large-scale software-defined networking where recursive partitioning improves scalability while keeping computation manageable.

**Work Growth Across Recursive Levels**

The recursion tree provides additional insight into how computation increases.

- **Level 0:** 1 subproblem, work = **n**
- **Level 1:** 3 subproblems, each of size **n/2**, work = **3(n/2) = 1.5n**
- **Level 2:** 9 subproblems, each of size **n/4**, work = **9(n/4) = 2.25n**
- **Level 3:** 27 subproblems, each of size **n/8**, work = **27(n/8) = 3.375n**

Each successive level performs approximately **1.5 times** more work than the previous level. Since the work increases geometrically toward the leaf nodes, the deepest recursive levels dominate the total running time, leading to the final complexity of $\boldsymbol{\Theta(n^{1.585})}$.

**Space Complexity Analysis**

Each recursive call stores:

- Local variables.
- Routing state information.
- Return addresses.

The recursion depth is

$$\log_2 n$$

because the problem size is halved at every recursive step.

Only one recursive path remains active at any instant due to depth-first execution, so the maximum recursion stack contains $\boldsymbol{\log_2 n}$ frames.

Therefore,

$$\boxed{\text{Space Complexity} = \Theta(\log n)}$$

The algorithm is memory efficient because it requires only logarithmic auxiliary memory despite performing multiple recursive calls.

**Final Result**

- Recurrence Relation: **T(n)=3T(n/2)+n**
- Master Theorem Parameters: **a = 3, b = 2, f(n)=n**
- Critical Function: **n^($\log_2$3)=n^1.585**
- Applicable Case: **Master Theorem Case 1**

- **Time Complexity: Θ(n^1.585)**
- **Space Complexity: Θ(log n)**

**Practical Significance in Network Routing Systems:**

In modern communication networks, routing optimization algorithms must efficiently determine the shortest or most reliable path among thousands or even millions of interconnected nodes. By recursively dividing the routing problem into smaller subnetworks, the algorithm can process large routing tables in parallel before integrating the partial solutions. This divide-and-conquer strategy significantly reduces computational overhead compared to examining every possible route individually.

The recurrence **$T(n) = 3T(n/2) + n$** models situations where each routing decision generates three independent optimization tasks, such as evaluating multiple routing policies, redundant communication paths, or alternative forwarding strategies. The linear merging phase represents operations such as updating routing tables, resolving path conflicts, synchronizing network topology information, and selecting the optimal route among the computed alternatives.

Although the running time **$\Theta(n^{1.585})$** is higher than linear, it remains substantially more efficient than quadratic algorithms **$\Theta(n^2)$** for large-scale networks. As the number of nodes increases, the algorithm scales effectively while maintaining acceptable response times, making it suitable for high-performance networking environments.

Overall, the recurrence demonstrates how recursive decomposition improves routing efficiency by balancing computational workload across smaller subproblems while keeping auxiliary memory usage minimal. This combination of sub-quadratic time complexity and logarithmic space complexity makes the algorithm practical for real-time routing optimization, dynamic traffic engineering, load balancing, fault-tolerant communication, and large-scale distributed networking systems.

**Q2. Distributed Data Processing System**

A distributed data processing system divides a large dataset into smaller blocks, processes each block independently, and combines the results to produce the final output. This divide-and-conquer strategy is common in distributed databases, cloud computing frameworks, parallel analytics engines, and big data platforms. The recurrence relation representing this algorithm is:

**T(n) = T(n/2) + n**

where:

- **n** = Number of data records or data blocks.
- **T(n/2)** = Time required to process one half of the data.
- **n** = Linear work performed for reading, processing, merging, or aggregating the results.

Unlike algorithms that branch into multiple recursive calls, this algorithm performs **only one recursive call** at each level while carrying out a linear amount of processing before or after the recursive computation.

**Recurrence Solving Method**

The recurrence can be solved using either the **Master Theorem** or the **Recursion Tree Method**. The recursion tree provides a clear understanding of how the work is distributed across recursive levels.

At each recursive level:

- **Level 0:** Work = **n**
- **Level 1:** Work = **n/2**
- **Level 2:** Work = **n/4**
- **Level 3:** Work = **n/8**
- …
- **Final Level:** Work = **1**

The total work becomes

$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \cdots + 1$$

This forms a geometric series.
Using the geometric series formula,

$$n\left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots\right) < 2n$$

Therefore,

$$T(n) = \Theta(n)$$

**Verification Using Master Theorem**

The recurrence matches the standard form

$$T(n) = aT(n/b) + f(n)$$

where:

- **a = 1**
- **b = 2**
- **f(n)=n**

Compute

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

Since

$$f(n) = n = \Omega(n^{0+\varepsilon})$$

for **ε = 1**, and the regularity condition is satisfied because

$$a\, f(n/b) = \frac{n}{2} \leq cn$$

for some constant **c < 1**, **Case 3 of the Master Theorem** applies.
Hence,

$$\boxed{T(n) = \Theta(n)}$$

The linear processing work dominates the recursive computation, making the overall algorithm linear in time.

**Growth of Work Across Recursive Calls**

The amount of work decreases by half at every recursive level because only one subproblem is generated. As the recursion progresses, the computational effort becomes smaller and smaller:

- Initial level performs the largest amount of work.
- Every subsequent level performs only half the work of the previous level.
- The total accumulated work converges to approximately **2n**, which is still proportional to **n**.

Unlike divide-and-conquer algorithms that generate multiple recursive branches, this recurrence forms a single recursive chain. Consequently, the algorithm scales efficiently even for very large datasets, making it suitable for distributed storage systems, cloud-based batch processing, log analysis, and streaming data aggregation.

**Space Complexity Analysis**

The recursion depth is determined by repeatedly halving the input size:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \ldots, 1$$

The number of recursive levels is

$$\log_2 n$$

Each recursive call occupies one activation record on the call stack, containing local variables and return information.
Since only one recursive call is active at any time, the maximum stack depth equals $\log_2 n$.
Therefore,

$$\boxed{\text{Space Complexity} = \Theta(\log n)}$$

This logarithmic memory usage makes the algorithm highly space efficient while maintaining linear execution time.

**Final Result**

- Recurrence Relation: **T(n)=T(n/2)+n**
- Solving Method: **Recursion Tree and Master Theorem**
- Master Theorem Parameters: **a = 1, b = 2, f(n)=n**
- Applicable Case: **Master Theorem Case 3**
- **Time Complexity: Θ(n)**
- **Space Complexity: Θ(log n)**